

AURA: Autonomous Understanding & Replanning Agent

Technical Summary Sheet

Team Name	Wall E Warriors
Team Members	Babitha Ravindra (2441616) • Chris Samuel (2441617) • Lidia Arul (2441630)
GitHub Repository	github.com/BabithaRavindra/AI---CIA-3---Hackathon

Overview

AURA is a knowledge-based navigation agent that plans and executes a route across a 20×20 grid sector populated with five obstacle types, replanning in real time when previously unseen hazards are perceived. The system combines a propositional-logic knowledge base for hazard reasoning with A* search for pathfinding, and presents the process through a Pygame visualization showing live search probing, collisions, and the final route.

System Architecture

The project is organized into three modules: an **AI / Decision Engine** (path planning, hazard reasoning, and replanning), a **Pygame interface** (grid rendering, obstacle visuals, and the live search-probe display), and a **knowledge and explainability layer** that surfaces the agent's reasoning to the user. This summary focuses on the AI / Decision Engine, contributed by Babitha Ravindra.

AI Engine (Decision & Reasoning Module)

Perception & Knowledge Base. A propositional-logic knowledge base (*PropositionalKB*) stores hazard and safety facts as clauses and answers queries by resolution refutation. On each step the agent perceives its immediate neighborhood and asserts facts such as $HazardSignal(x,y) \Rightarrow \neg Safe(x,y)$, building an explicit, queryable model of the sector rather than relying on the raw grid alone.

Path Planning. Route search uses A* with a Manhattan-distance heuristic and unit step cost. Nodes are expanded in order of $f = g + h$, and the path is reconstructed by walking the *came_from* chain back to the start once the goal is reached.

Dynamic Replanning. Before each move the agent checks whether the next planned cell is still safe. If a cell has become blocked or is newly entailed as a hazard, the agent enters a *REPLANNING* state, increments a replan counter, and reruns the search from its current position rather than continuing along a stale path.

State & Metrics Tracking. The agent exposes an explicit state machine (IDLE, PLANNING, ACTING, REPLANNING, REACHED_GOAL, FAILED) and a metrics object tracking path cost, nodes expanded, replan count, and collisions avoided, so planning behavior can be inspected during a run.

Integration Hooks. The engine exposes *plan()*, *step()*, and *reset()* as the interface consumed by the Pygame front end, keeping the reasoning logic independent of the rendering and interaction code.

Tech Stack

Python, Pygame (visualization), propositional logic / resolution-based reasoning, A* search.